



Module Code & Module Title

CS4001NT Programming

Assessment Weightage & Type

Milestone 1 (Week 18)

Year & Semester

Autumn 2025

Student Name: Anusha Bishwas

London Met ID: 25028734

College ID: NP05CP4A250031

Assignment Due Date: Monday, March 9, 2026

Assignment Submission Date: Monday, March 9, 2026

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1. Introduction.....	1
1.1. Objectives.....	1
1.2. Development Environment – BlueJ.....	2
1.2.1 Why BlueJ?	2
1.3. Scope and Limitations	2
2. Technical Documentation.....	3
2.1. Class Diagram.....	3
2.2. Pseudocode	4
2.2.1 calculateTokenUsage (Parent Class).....	4
2.2.2 usePrompt (PersonalPlan)	4
2.2.3 usePrompt (ProPlan).....	5
2.3. Method Description	5
2.3.1 AIModel Class.....	5
2.3.2. PersonalPlan Class	8
2.3.3 ProPlan Class	10
2.3.4 Main Class	13
3. Testing & Evaluation.....	15
3.1. Testing Evidence	15
3.1.1 Test 1: Valid Prompt execution (Tokens within Context)	15
3.1.2. Test 2: Prompt rejected due to Context Limit (Input + Output + System > Context)	16
3.1.3. Test 3 (Personal): Exhausting Monthly Quota (Count reaches 0) and purchasing more	17
3.1.4. Test 4 (Pro): Verifying unlimited prompts (No count decrement)	18

3.2. Error Correction	18
Error 1- Syntax Error: Missing Semicolon	18
Error 2 – Runtime Error: Token Sum Miscalculation	19
Error 3 – Compile Error: Incorrect super() Call in PersonalPlan	20
3.3. Conclusion	21
Use of Abstraction in Token Validation	21
4. Appendix	22
4.1. AIModel	22
4.2. PersonalPlan	23
4.3. ProPlan	26
4.4. Main	29
5. References	33

Table of Figures

Figure 1.: BlueJ	2
Figure 2: UML Class Diagram AIModel (Abstract class) with PersonalPlan and ProPlan subclasses	3
Figure 3: AIModel.calculateTokenUsage(inputTokens, outputTokens)	4
Figure 4: PersonalPlan.usePrompt (inputTokens, outputTokens)	4
Figure 5: ProPlan.usePrompt (inputTokens, outputTokens).....	5
Figure 6: Console Output- Test 1	16
Figure 7: PersonalPlan-Prompt Rejected	17
Figure 8:ProPlan prompt rejected.....	17
Figure 9: Console Output-Test 3	17
Figure 10:Console Output-Test 4	18
Figure 11: Code with Fault	19
Figure 12: Error, Missing semicolon	19
Figure 13: Corrected code, Semicolon added	19
Figure 14: Code with Fault	19
Figure 15: Corrected Code	19
Figure 16: Code with Fault	20
Figure 17: Error shown by Compiler.....	20
Figure 18: Corrected Code	20

Table of Tables

Table 1: Method Descriptions and Functional Overview of the AIModel Class	5
Table 2: Method Descriptions and Functional Overview of the PersonalPlan class	8
Table 3: Method Descriptions and Functional Overview of the ProPlan Class	10
Table 4: Method Description and Functional Overview of Main Class.....	13
Table 5: Test 1- Input	15
Table 6: Test 2 - Input	16

1. Introduction

This report presents the design, implementation, and testing of a console-based AI Subscription Management System developed for the CS4001NT Programming Milestone 1 coursework. The system is implemented using Java and demonstrates key Object-Oriented Programming (OOP) concepts such as Abstraction, Inheritance, Encapsulation, and Polymorphism.

The system includes two subscription types:

- i. PersonalPlan, designed for individual users,
- ii. ProPlan, designed for professional teams.

Both plans inherit common AI model properties from an abstract parent class called AIModel. The core of this system is to check if the number of tokens used in a request is within each of the model's context window. In addition, the PersonalPlan limits on how many prompts a user can send each month by applying a monthly prompt quota.

1.1. Objectives

- To design an abstract parent class (AIModel) that stores AI model information and handles token validation.
- To implement a PersonalPlan class that checks both monthly prompt limit and the context window.
- To implement a ProPlan class that allows unlimited prompts but still checks the context window limit.
- To demonstrate the program's functionality using a Main class through console output.
- To provide clear documentation and testing results for the system.

1.2. Development Environment – BlueJ



Figure 1.: BlueJ

This project was developed and tested using BlueJ, an integrated Java environment specifically designed for introductory teaching. BlueJ is based on the Blue System. Blue is an integrated teaching environment and language, developed at the University of Sydney and Monash University, Australia.

BlueJ is a Windows-based platform for the Java Development Kit (JDK). It is a free Java environment started in 1999 by Michael Kolling and John Rosenberg at Monash University, Australia, as a successor to Blue. It was developed to support the learning and teaching of OOP (object-oriented programming).

1.2.1 Why BlueJ?

In particular, BlueJ creates an environment that emphasises classes and objects as its basic units of interaction, so that students are naturally led into a way of thinking in terms of classes and objects. BlueJ is sophisticated, but at the same time easy to use, so that students can start using it straight away without needing any long introduction. BlueJ uses visualisation to display class structure. Structuring problem is one of the most important aspects of object-oriented programming.

1.3. Scope and Limitations

For Milestone 1, the graphical user interface (GUI) component SubscriptionGUI.java is not included as advised. All the features of the program are demonstrated via console output using the Main class.

2. Technical Documentation

2.1. Class Diagram

The Unified Modeling Language (UML) diagram shows the hierarchical relationship between the abstract parent class and its two subclasses. Both PersonalPlan and ProPlan extend the abstract class, inheriting the calculateTokenUsage() method while providing their own usePrompt() and display() implementations. This visual representation displays the inheritance structure and highlights the key attributes and methods that define the system's functionality.

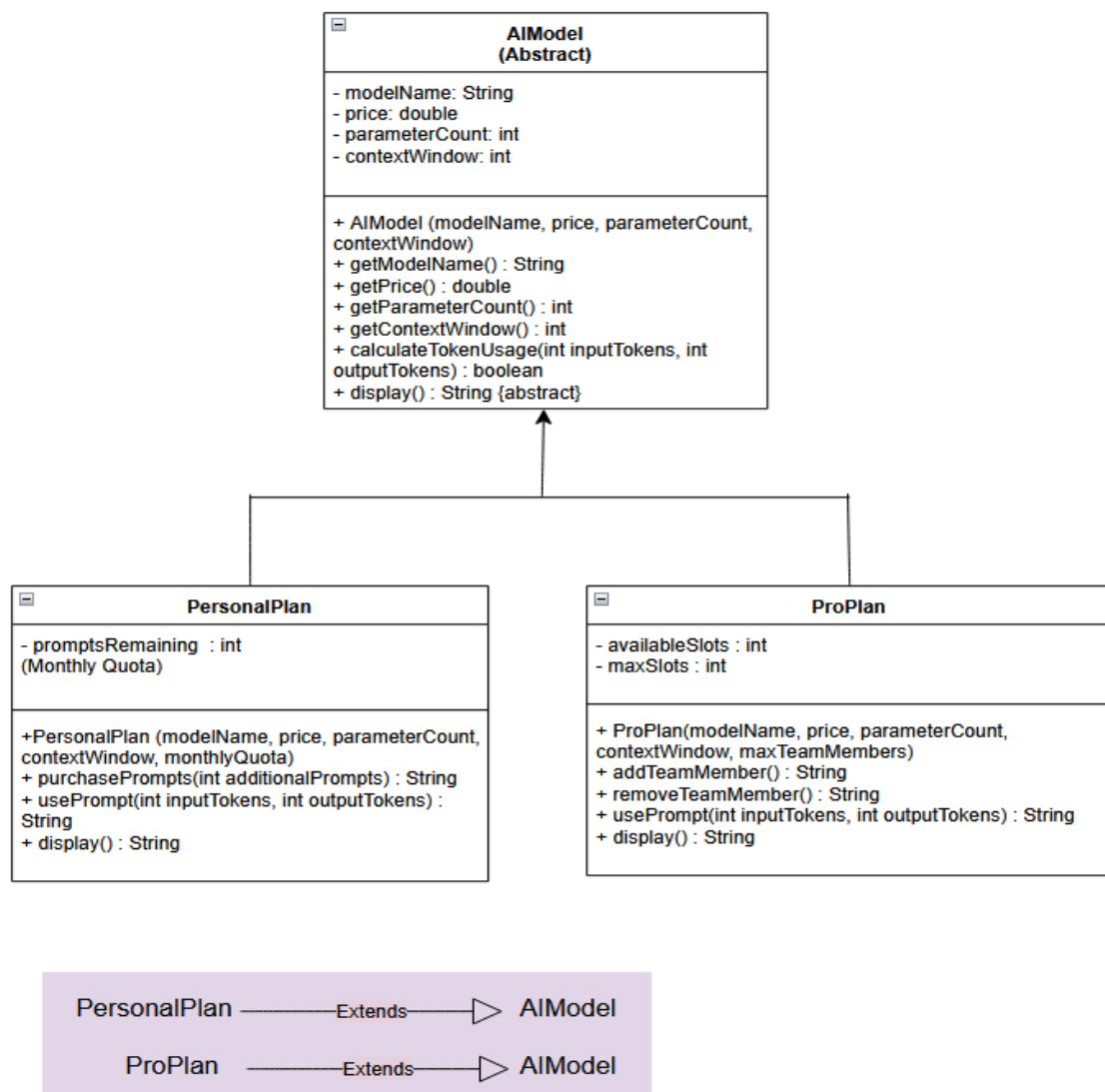


Figure 2: UML Class Diagram AIModel (Abstract class) with PersonalPlan and ProPlan subclasses

2.2. Pseudocode

The following pseudocode describes the three core algorithms: the shared token validation method in AIModel, and the usePrompt() logic for each subclasses.

2.2.1 calculateTokenUsage (Parent Class)

It ensures that the total token does not exceed the model's context window.

```
// Method to validate token usage before a prompt is processed
// It ensures the total tokens do not exceed the model's context window
public boolean calculateTokenUsage(int inputTokens, int outputTokens) {
    int systemTokens = 50;    // fixed overhead tokens used internally by the system

    // Total tokens used in a request (input + output + system tokens)
    int totalTokens = inputTokens + outputTokens + systemTokens;

    // Returns true if request is within the context limit
    return totalTokens <= contextWindow;
}
```

Figure 3: AIModel.calculateTokenUsage(inputTokens, outputTokens)

The system adds up inputTokens, outputTokens, and a fixed systemTokens that is fixed internally by the system. It checks if the total fits within the model's contextWindow. If yes, allows the request, if no, rejects it. This ensures if the model can handle the prompt.

2.2.2 usePrompt (PersonalPlan)

It allows prompt usage only if quota is available and token limits are respected.

```
// Simulates sending a prompt to the AI model
public String usePrompt(int inputTokens, int outputTokens) {
    // Check if the user has exhausted their monthly prompt quota
    if (promptsRemaining <= 0) {
        return "Error: Monthly quota exhausted.";
    }

    // Validate token usage using the inherited method from AIModel
    if (!calculateTokenUsage(inputTokens, outputTokens)) {
        return "Error: Context limit exceeded.";
    }

    // Reduce available prompts after successful usage
    promptsRemaining--;
    return "Prompt used successfully. \n-Remaining: " + promptsRemaining;
}
```

Figure 4: PersonalPlan.usePrompt (inputTokens, outputTokens)

First, it checks the monthly promptRemaining. If none are left, it stops with "Monthly quota exhausted." Next, it calls the token check. If the total tokens exceed the context, it stops with "Context limit exceeded." Otherwise, it decreases the quota by 1 and confirms the prompt was used.

2.2.3 usePrompt (ProPlan)

It allows unlimited prompts but still validates token limits.

```
// ProPlan allows unlimited prompts but still checks token limits
public String usePrompt(int inputTokens, int outputTokens) {
    // Validate token usage using the inherited method from AIModel
    if (!calculateTokenUsage(inputTokens, outputTokens)) {
        return "Error: Context limit exceeded.\n";
    }
}
```

Figure 5: ProPlan.usePrompt (inputTokens, outputTokens)

There's no monthly quota for this model. The system only checks token usage. If the context is exceeded, it rejects. If it is not, it processes successfully. This allows unlimited prompts within the model's capacity.

2.3. Method Description

2.3.1 AIModel Class

Table 1: Method Descriptions and Functional Overview of the AIModel Class

Method Name	Return Type	Parameters	Internal Working
AIModel(...)	None (constructor)	modelName:String price: double parameterCount: int contextWindow: int	• The constructor is called when a new AI model object is created. • It receives the model's name, price, parameter count, and context window as input. • The values are assigned to the class's private variables:

			modelName, price, parameterCount, contextWindow. • This initializes the AI model with the required configuration. • The context window stored here is later used to validate token usage in prompts.
getModelName()	String	None	• The method is called when another class needs the AI model's name. • It accesses the private variable modelName. • The value of modelName is returned to the caller.
getPrice()	Double	None	• The method retrieves the price of the AI model. • It accesses the private variable price. • The price (in NPR per 1 lakh tokens) is returned.
getParameterCount()	Int	None	• The method accesses the parameterCount variable. • This value represents the size of the AI model in billions of parameters. • The value is returned to the caller.
getContextWindow()	int	None	• The method retrieves the maximum token capacity of the AI model.

			• It accesses the variable contextWindow. • The value returned represents the maximum number of tokens allowed in one request.
calculateTokenUsage()	boolean	inputTokens: int outputTokens: int	• The system defines a fixed system token overhead of 50 tokens. • The method calculates the total token usage using the formula: Total Tokens = Input Tokens + Output Tokens + System Tokens • The calculated value is compared against the contextWindow limit. • IF total tokens are within the limit, the method returns true. • If the total tokens exceed the context window, the method returns false.
display()	String (abstract)	None	• This method is declared as abstract, meaning it has no implementation in this class. • Any subclass (such as PersonalPlan or ProPlan) must implement this method. • The method is used to display plan details, such as: model

			name, price, parameters, context window, plan-specific features (like prompt quota or team slots).
--	--	--	--

2.3.2. PersonalPlan Class

Table 2: Method Descriptions and Functional Overview of the PersonalPlan class

Method Name	Return Type	Parameters	Internal Working
PersonalPlan(...)	None (constructor)	modelName: String price: double parameterCount: int contextWindow: int monthlyQuota: int	• The constructor is executed when a new PersonalPlan object is created. • It first calls the parent class constructor using <code>super(...)</code> to initialize share AI model attributes: model name, price, parameter count, context window. • The constructor then validates the monthly prompt quota. • If monthly Quota is negative, it is automatically set to 0 to prevent invalid values. • The validated quota is stored in <code>promptsRemaining</code>. • This value represents how many prompts the user can still send during the month.
purchasePrompts()	String	additionalPrompts: int	• The method is called when the user wants to buy extra prompts. • It receives the number of prompts the user wants to add (additionalprompts).

			• A validation check makes sure that the value is greater than 0. • If the value is 0 or negative, The method returns: "Invalid amount. Must be positive." • If valid the additional prompts are added to the existing quota. • The system then return a confirmation message showing how many prompts were added and the new total prompts available.
usePrompt()	String	inputTokens: int outputTokens: int	• The method first checks the monthly prompt quota stored in promptsRemaining. • If the quota has reached 0, the request is rejected. • If prompts remain, the method calls calculateTokenUsage() to verify token limits. • If the token usage exceeds the context window, the request is rejected. • If both conditions pass, then the prompt is executed and promptsRemaining is decreased by 1.
display()	String	None	• This method overrides the abstract display() method from the AIModel parent class. • It gathers information about the Personal Plan using inherited getter

			methods: getModelName(), getPrice(), getParameterCount(), getContextWindow(). • It also includes the plan-specific attribute: promptsRemaining. • All these values are formatted into a readable string showing the full plan details. • The formatted string is returned and displayed in the console.
--	--	--	--

2.3.3 ProPlan Class

Table 3: Method Descriptions and Functional Overview of the ProPlan Class

Method Name	Return Type	Parameters	Internal Working
ProPlan(...)	None (constructor)	modelName: String price: double parameterCount: int contextWindow: int maxTeamMemmmbers: int	• The constructor runs when a new ProPlan object is created. • It first calls the parent class constructor using super(...) to initialize shared AI model attributes: model name, price, parameter count, context window. • The value maxTeaenMembers is stored as maxSlots.

			• availableSlots is initialized with the same value, representing the current number of team member spaces available.
addTeamMember()	String	None	• The method checks if any team slots are available. • If availableSlots ≤ 0, the system prevents adding more members and returns an error message. • If slots are available, availableSlots is decreased by one. • A confirmation message is returned showing the remaining slots.
removeTeamMember()	String	None	• The method checks if all slots are already available. • If true, no members exist to remove and the system returns a message. • Otherwise, availableSlots is increased by one. • This frees a slot and the updated slot count is returned.
usePrompt	String	inputTokens: int outputTokens: int	• The method calls calculateTokenUsage().

			• If the token usage exceeds the context window, the request is rejected. • Unlike PersonalPlan, Proplan does not track prompt count. • Therefore, prompts can be used unlimited times as long as token limits are respected. • The system returns a success message if the request is valid.
display()	String	None	• This method overrides the abstract display() method from the parent class. • It retrieves model information using getter methods: model name, price, parameter count, context window. • It also includes the available team slots. • The details are formatted into a readable string and returned.

2.3.4 Main Class

Table 4: Method Description and Functional Overview of Main Class

Method Name	Return Type	Parameters	Internal Working
Main	void	String[] args	• The program starts execution from the main method. • Objects for PersonalPlan and ProPlan are created with their respective model, details, context windows, quotas, and team slots. • The system header and PersonalPlan section title are printed to the console. • The display() method is called to show the PersonalPlan details. • A loop simulated prompt usage three times using usePrompt(), reducing the prompt quota each time. • Another prompt attempt is made to demonstrate the quota exhausted validation. • The purchasePrompts() method is called to add additional prompts to the PersonalPlan. • A prompt is used again to confirm that the quota has increased. • The ProPlan section is displayed and its details are printed using display(). • A prompt is processed using usePrompt() to demonstrate token validation without quota limits. • Another prompt exceeding the context window is attempted to demonstrate the context limit error.

			• Team member management is demonstrated using <code>addTeamMember()</code> and <code>removeTeamMember()</code>. • Finally, the update ProPlan details are displayed using <code>display()</code>.
--	--	--	--

3. Testing & Evaluation

All four test scenarios were executed through the Main class console runner. The PersonalPlan was set with a context window of 200 tokens and a quota of 3 prompts. The ProPlan was set with a context window of 500 tokens and 3 team slots. Each of the test's expected behaviour, analysis, and actual output are presented below.

3.1. Testing Evidence

Screenshots of Console Output demonstrating each of the tests are presented below.

3.1.1 Test 1: Valid Prompt execution (Tokens within Context)

Objective: To verify that a prompt whose total tokens (input + output + system overhead) fall within the context window is processed successfully.

Table 5: Test 1- Input

Parameter	Value
Plan	PersonalPlan ("Turbo")
Input Tokens	100
Output Tokens	50
System Overhead	50(fixed)
Total Tokens	$100 + 50 + 50 = 200$
Context Window	200
Expected Result	True -> "Prompt used successfully"

Expected Result: Prompt executes successfully.

3.1.1.1. Console Output

```
Using prompts:
-Prompt used successfully.
-Remaining: 2
-Prompt used successfully.
-Remaining: 1
-Prompt used successfully.
-Remaining: 0
```

Figure 6: Console Output- Test 1

Each call to usePrompt(100, 50) resulted in total of 200 tokens, exactly meeting the 200-token context window. The quota decremented correctly from 3 to 2 to 1 to 0 after three successful calls.

3.1.2. Test 2: Prompt rejected due to Context Limit (Input + Output + System > Context)

Objective: To verify a prompt whose total tokens exceed the context window is correctly rejected for both plan types.

Table 6: Test 2 - Input

Parameter	PersonalPlan Test	ProPlan Test
Input Tokens	200	400
Output Tokens	100	200
System Overhead	50	50
Total Tokens	350	650
Context Window	200	500
Result	Context limit exceeded	Context limit exceeded

Expected Result: Prompt request is rejected due to context limit.

3.1.2.1. Console Output

```
Attempting another prompt:  
-Error: Monthly quota exhausted.
```

Figure 7: PersonalPlan-Prompt Rejected

```
Using a prompt (input + output exceeds context):  
- Error: Context limit exceeded.
```

Figure 8: ProPlan prompt rejected

calculateTokenUsage() returned false for both scenarios. The shared parent method prevented duplication, the same logic in AIModel validated both subclasses without repetition.

3.1.3. Test 3 (Personal): Exhausting Monthly Quota (Count reaches 0) and purchasing more

Objective: To exhaust the PersonalPlan monthly quota (3 prompts), verify the "Monthly quota exhausted" error, then purchase additional prompts and confirm recovery.

4.1.3.1. Console Output

```
Attempting another prompt:  
-Error: Monthly quota exhausted.  
  
Buying more prompts:  
- 3 prompts added Successfully.  
-Total: 3  
|  
Using a prompt after purchase:  
- Prompt used successfully.  
-Remaining: 2
```

Figure 9: Console Output-Test 3

The quota check fires before the token check, ensuring a clear error when the monthly quota is exhausted. After `purchasePrompts(3)` successfully topped up the quota to 3, the subsequent `usePrompt()` call decremented it to 2, confirming correct recovery.

3.1.4. Test 4 (Pro): Verifying unlimited prompts (No count decrement)

Objective: To confirm that ProPlan does not track any prompt count, and only enforces the context window constraint.

Console Output

```
Using a prompt:
- Prompt used successfully (unlimited for ProPlan).

Managing team members:
- Team member added.
-Remaining slots: 2
- Team member added.
-Remaining slots: 1
- Team member added.
-Remaining slots: 0
- Team member removed. Available slots: 1

Final ProPlan Status:
ProPlan Model: Turbo++
Price: NPR.5000.0
Parameters: 6B
Context Window: 500
Available Team Slots: 1
```

Figure 10: Console Output-Test 4

ProPlan's `usePrompt()` succeeded without touching any counter. Team slot management worked correctly, slots decremented on add, incremented on remove. The boundary check prevented over-removal.

3.2. Error Correction

Three distinct error types were deliberately introduced and resolved during development. Each is documented with the code, the error and the correct action.

Error 1- Syntax Error: Missing Semicolon

Location: `AIModel.Java` – `calculateTokenUsage()` method

```
// Total tokens used in a request (input + output + system tokens)
int totalTokens = inputTokens + outputTokens + systemTokens
// Returns true if request is within the context limit
return totalTokens <= contextWindow;
```

Figure 11: Code with Fault

```
// Total tokens used in a request (input + output + system tokens)
int totalTokens = inputTokens + outputTokens + systemTokens
// Returns true if request is within the context limit
return totalTokens <= contextWindow;
```

'; expected

Figure 12: Error, Missing semicolon

```
// Total tokens used in a request (input + output + system tokens)
int totalTokens = inputTokens + outputTokens + systemTokens;
// Returns true if request is within the context limit
return totalTokens <= contextWindow;
```

Figure 13: Corrected code, Semicolon added

Java requires every statement to end with a semicolon. The Java compiler immediately flags the missing token and stops compilation. The fix was adding the semicolon at the end of the variable declaration line.

Error 2 – Runtime Error: Token Sum Miscalculation

Location: AIModel.java – calculateTokenUsage() method

```
int systemTokens = 50;    // fixed overhead tokens used internally by the system

// Total tokens used in a request (input + output + system tokens)
int totalTokens = inputTokens + outputTokens - systemTokens;

// Returns true if request is within the context limit
return totalTokens <= contextWindow;
```

Figure 14: Code with Fault

```
int systemTokens = 50;    // fixed overhead tokens used internally by the system

// Total tokens used in a request (input + output + system tokens)
int totalTokens = inputTokens + outputTokens + systemTokens;

// Returns true if request is within the context limit
return totalTokens <= contextWindow;
```

Figure 15: Corrected Code

The system token overhead (50) represents fixed processing cost and must be added to the request tokens, not subtracted, using subtraction caused the system to underestimate total token usage, accepting requests that should have been rejected. No compile error occurs, this is a logic error discovered through careful test case analysis.

Error 3 – Compile Error: Incorrect super() Call in PersonalPlan

Location: PersonalPlan.java – constructor

```
public PersonalPlan(String modelName, double price, int parameterCount, int contextWindow, int monthlyQuota) {
    super(modelName, price, parameterCount, contextWindow, monthlyQuota); //call parent constructor

    //Validate monthly quota to prevent negative values
    if(monthlyQuota<0){
        monthlyQuota = 0;
    }
    this.promptsRemaining = monthlyQuota;
}
```

Figure 16: Code with Fault

```
public PersonalPlan(String modelName, double price, int parameterCount, int contextWindow, int monthlyQuota) {
    super(modelName, price, parameterCount, contextWindow, monthlyQuota); //call parent constructor

    //Validate monthly quota to prevent negative values
    if(monthlyQuota<0){
        monthlyQuota = 0;
    }
    this.promptsRemaining = monthlyQuota;
}
```

constructor AIModel in class AIModel cannot be applied to given types;
required: java.lang.String,double,int,int
found: java.lang.String,double,int,int,int
reason: actual and formal argument lists differ in length

Figure 17: Error shown by Compiler

```
public PersonalPlan(String modelName, double price, int parameterCount, int contextWindow, int monthlyQuota) {
    super(modelName, price, parameterCount, contextWindow); //call parent constructor

    //Validate monthly quota to prevent negative values
    if(monthlyQuota<0){
        monthlyQuota = 0;
    }
    this.promptsRemaining = monthlyQuota;
}
```

Figure 18: Corrected Code

The parent class AIModel's constructor only accepts 4 parameters. The monthlyQuota is a PersonalPlan-specific attribute and should be handled exclusively in the child class. Passing it to super() violated the constructor signature, causing a compile-time error. The fix was removing monthlyQuota from the super() call and assigning it directly to this.promptsRemaining in the child constructor.

3.3. Conclusion

This milestone successfully demonstrates the use of core Object-Oriented Programming principles to build a structured AI subscription Management System in Java. The project shows how abstraction and inheritance help simplify the design of an AI subscription system.

Use of Abstraction in Token Validation

Placing the `calculateTokenUsage()` method in the abstract parent class `AIModel` was the most significant architectural benefit. Both the `PersonalPlan` and `ProPlan` would have independently re-implemented the same token arithmetic without abstraction. Code duplication was entirely eradicated, the logic was written only once. Maintenance became easy, changing the system token overhead require editing only one class. Consistency was guaranteed, both plans always use the same validation rule. The abstract `display()` method made sure that each subclass provided its own relevant details, enforced at compile time.

4. Appendix

4.1. AIModel

```
// Abstract parent class representing a generic AI model

// This class stores common properties shared by all AI subscription plans

public abstract class AIModel {

    private String modelName;    //Name of the AI Model

    private double price;        //Cost in NPR per 1 Lakh tokens

    private int parameterCount;  // Size of the AI model(in billions of parameters)

    private int contextWindow;   // maximum number of tokens allowed per request


    // Constructor used to initialize the AI model properties

    public AIModel(String modelName, double price, int parameterCount, int
contextWindow) {

        this.modelName = modelName;

        this.price = price;

        this.parameterCount = parameterCount;

        this.contextWindow = contextWindow;

    }


    // Getter methods provide controlled access to private/protected data

    // These follow the encapsulation principle in Object-Oriented Programming

    public String getModelName() { return modelName; }
```

```
public double getPrice() { return price; }
```

```
public int getParameterCount() { return parameterCount; }
```

```
public int getContextWindow() { return contextWindow; }
```

```
// Method to validate token usage before a prompt is processed
```

```
// It ensures the total tokens do not exceed the model's context window
```

```
public boolean calculateTokenUsage(int inputTokens, int outputTokens) {
```

```
    int systemTokens = 50; // fixed overhead tokens used internally by the system
```

```
    // Total tokens used in a request (input + output + system tokens)
```

```
    int totalTokens = inputTokens + outputTokens + systemTokens;
```

```
    // Returns true if request is within the context limit
```

```
    return totalTokens <= contextWindow;
```

```
}
```

```
// Abstract method that must be implemented by subclasses
```

```
// Each plan will display its details in its own format
```

```
public abstract String display();
```

```
}
```

4.2. PersonalPlan

```
// PersonalPlan class represents an individual AI subscription plan
```

```
// It inherits common model properties and behaviour from the AIModel class

public class PersonalPlan extends AIModel {

    // Stores the number of prompts the user can still use this month

    private int promptsRemaining;

    // Constructor initializes the personal plan along with the monthly prompt quota

    public PersonalPlan(String modelName, double price, int parameterCount, int
contextWindow, int monthlyQuota) {

        super(modelName, price, parameterCount, contextWindow); //call parent
constructor

        //Validate monthly quota to prevent negative values

        if(monthlyQuota<0){

            monthlyQuota = 0;

        }

        this.promptsRemaining = monthlyQuota;

    }

    // Allows the user to purchase additional prompts when their quota runs low

    public String purchasePrompts(int additionalPrompts) {

        // Validation to ensure the user enters a valid positive number
```

```
        if (additionalPrompts <= 0) {  
            return "Invalid amount. Must be positive.";  
        }  
  
        // Increase the remaining prompts  
        promptsRemaining += additionalPrompts;  
  
        return additionalPrompts + " prompts added Successfully. \n-Total: " +  
promptsRemaining;  
    }  
  
    // Simulates sending a prompt to the AI model  
    public String usePrompt(int inputTokens, int outputTokens) {  
        // Check if the user has exhausted their monthly prompt quota  
        if (promptsRemaining <= 0) {  
            return "Error: Monthly quota exhausted.";  
        }  
  
        // Validate token usage using the inherited method from AIModel  
        if (!calculateTokenUsage(inputTokens, outputTokens)) {  
            return "Error: Context limit exceeded.";  
        }  
  
        // Reduce available prompts after successful usage  
        promptsRemaining--;
```

```
        return "Prompt used successfully. \n-Remaining: " + promptsRemaining;
    }

    // Displays the details of the PersonalPlan subscription
    @Override
    public String display() {
        return "PersonalPlan Model: " + getModelName() +
            "\nPrice: NPR. " + getPrice() +
            "\nParameters: " + getParameterCount() + "B" +
            "\nContext Window: " + getContextWindow() +
            "\nMonthly Prompts Remaining: " + promptsRemaining;
    }
}
```

4.3. ProPlan

```
// ProPlan represents a professional AI subscription plan

// It inherits common AI model properties from the AIModel parent class
public class ProPlan extends AIModel {

    // Tracks the number of team member slots available in the plan
    private int availableSlots;

    //Stores the maximum allowed team members
    private int maxSlots;
```

```
// Constructor initializes the ProPlan with model details and maximum team members

public ProPlan(String modelName, double price, int parameterCount, int
contextWindow, int maxTeamMembers) {

    super(modelName, price, parameterCount, contextWindow);// call parent
constructor

    this.availableSlots = maxTeamMembers;

    this.maxSlots = maxTeamMembers;

}

// Adds a team member if there are available slots

public String addTeamMember() {

    // Prevent adding members when no slots are left

    if (availableSlots <= 0) {

        return "No available slots to add a member.";

    }

    // Reduce available slots after adding a member

    availableSlots--;

    return "Team member added. \n-Remaining slots: " + availableSlots;

}
```

```
// Removes a team member and frees up a slot

public String removeTeamMember() {

    //Prevent exceeding the maximum number of slots

    if(availableSlots >= maxSlots){

        return "All slots are available.";

    }

    availableSlots++;

    return "Team member removed. Available slots: " + availableSlots;

}


// ProPlan allows unlimited prompts but still checks token limits

public String usePrompt(int inputTokens, int outputTokens) {

    // Validate token usage using the inherited method from AIModel

    if (!calculateTokenUsage(inputTokens, outputTokens)) {

        return "Error: Context limit exceeded.\n";

    }

    // No prompt quota restriction for ProPlan users

    return "Prompt used successfully (unlimited for ProPlan).";

}
```

```
// Displays the details of the ProPlan subscription

@Override

public String display() {

    return "ProPlan Model: " + getModelName() +

        "\nPrice: NPR." + getPrice()+

        "\nParameters: " + getParameterCount() + "B" +

        "\nContext Window: " + getContextWindow() +

        "\nAvailable Team Slots: " + availableSlots;

}

}
```

4.4. Main

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        // Create objects for both subscription plans

        PersonalPlan personal = new PersonalPlan("Turbo", 1000, 2, 200, 3);

        ProPlan pro = new ProPlan("Turbo++", 5000, 6, 500, 3);

        // StringBuilder used to efficiently construct formatted output

        StringBuilder output = new StringBuilder();

    }

}
```

```
// === Unified Demo Panel ===

// Displays the header for the AI subscription system
System.out.println("\n      AI SUBSCRIPTION PLANS      \n\n");
System.out.println("-----Personal Plan-----\n");

// Display PersonalPlan details
System.out.println(personal.display());

// Demonstrates prompt usage within the PersonalPlan quota
System.out.println("\nUsing prompts:");
for (int i = 0; i < 3; i++) {
    System.out.println("-" + personal.usePrompt(100, 50));
}

// Attempt to use a prompt after the quota has been exhausted
System.out.println("\nAttempting another prompt: ");
System.out.println("-" + personal.usePrompt(100, 50));

// Demonstrates purchasing additional prompts
System.out.println("\nBuying more prompts:");
System.out.println("- " + personal.purchasePrompts(3));

// Using a prompt after increasing the quota
System.out.println("\nUsing a prompt after purchase:");
```

```
System.out.println("- " + personal.usePrompt(100, 50));

// === ProPlan Demonstration ===

System.out.println("\n-----Pro Plan-----\n");

// Display ProPlan details

System.out.println(pro.display());

// Demonstrates prompt usage for ProPlan (unlimited prompts)

System.out.println("\nUsing a prompt:");

System.out.println("- " + pro.usePrompt(300, 150));

// Demonstrates context window validation failure

System.out.println("\nUsing a prompt (input + output exceeds context:");

System.out.println("- " + pro.usePrompt(400, 200)); // total 600, but contextWindow
is 500

// Demonstrates team member management

System.out.println("Managing team members:");

System.out.println("- " + pro.addTeamMember());

System.out.println("- " + pro.addTeamMember());

System.out.println("- " + pro.addTeamMember());

System.out.println("- " + pro.removeTeamMember());
```

```
        // Display final state of the ProPlan after modifications  
        // Print the complete formatted output to the console  
        System.out.println("\nFinal ProPlan Status:");  
        System.out.println(pro.display());  
    }  
}
```

5. References

- [1] Weebly, M. Kolling and J. Rosenberg, "<https://bluej.weebly.com/about-bluej.html>," 5 December 2007. [Online]. [Accessed 1 March 2026].
- [2] W. bluej, "<https://bluej.weebly.com/about-bluej.html>," 5 December 2007. [Online]. Available: <https://www.weebly.com/>. [Accessed 2 March 2026].
- [3] GeeksforGeeks, "GeeksforGeeks," 15 July 2025. [Online]. [Accessed 2 March 2026].
- [4] Michael Kölling and John Rosenberg, *Blue - A Language for Teaching Object-Oriented Programming*, in Proceedings of 27th SIGCSE Technical Symposium on Computer Science Education, ACM, Philadelphia, Pennsylvania, 190-194, March 1996.
- [5] Michael Kölling and John Rosenberg, *An Object-Oriented Program Development Environment for the First Programming Course*, in Proceedings of 27th SIGCSE Technical Symposium on Computer Science Education, ACM, Philadelphia, Pennsylvania, 83-87, March 1996.